Entrega 1 · Arquitectura Big Data local (MITMA)

Oct 20, 2026 · @Aimar

Pipeline Big Data local (HDFS + YARN + Spark en Docker) sobre una semana de la matriz diaria de viajes entre municipios de MITMA: del 26-10-2022 al 01-11-2022, 78,3 millones de filas y 6,9 GB de CSV. Todo se reproduce con `docker compose up` y un único punto de entrada.

Resumen y cambios respecto a la inscripción

El pipeline sigue un único flujo:

- descarga los 7 ficheros diarios de MITMA;
- los guarda **tal cual en la zona raw de HDFS**;
- los limpia con Spark y un **esquema explícito**, enviando las filas inválidas a cuarentena;
- escribe la zona **curated en Parquet particionado por fecha**;
- responde **tres preguntas analíticas**;
- repite una agregación en **MapReduce (Hadoop Streaming) y con RDDs**;
- entrena una **regresión lineal con MLlib**.

Todo se lanza con `docker compose up` y un único punto de entrada, y cada cifra de este documento sale de `evidencias/` en el repositorio.

Elemento	Inscripción	Entrega 1	Motivo
Periodo	2 meses	**1 semana**: mié 26-10-2022 a mar 01-11-2022	Volumen manejable en el entorno local (~7 GB)
Tabla principal	Matriz diaria de viajes por municipio	Igual	—
Tablas auxiliares	Zonificación de municipios	Zonificación + **provincias** (INE) + **calendario** (festivos)	Nombres de provincia y tipo de día
Columna de partición	`fecha`	`fecha`	—

Elemento	Inscripción	Entrega 1	Motivo
P1 (agrupación)	Media diaria por provincia y día de la semana	Media diaria por provincia y **tipo de día**	Con 1 semana, n = 1 por día de la semana
P2 (ranking)	Top 3 municipios que más viajes reciben cada día	Igual	—
P3 (comparación con el grupo)	Desviación de cada municipio-día respecto a su media	Igual, respecto a la media de **la semana**, + variación respecto al día anterior (`lag`)	La media de 2 meses no existe
ML	Regresión de `viajes`	Regresión lineal: viajes medios diarios ~ municipio de origen + edad	Modelo simple, sin variables de tiempo

A.1 Las 5V aplicadas al dataset

De las cinco V, **Veracidad** y **Valor** aplican de lleno, **Volumen** aplica en parte, y **Velocidad** y **Variedad** no aplican en nuestro caso.

V	Cifras del dataset	¿Aplica?	Por qué
Volumen	6,87 GB de CSV (0,87 GB en `.gz`), **78.340.906 filas** en 7 días (~11,2 M/día)	**En parte**	Una semana cabe en un portátil: con un motor columnar en una sola máquina se procesaría. Pero el caso de uso (A.2) es el histórico: un año son ~4.000 M de filas y ~360 GB de CSV, que ya no caben en la memoria de un equipo. La arquitectura está pensada para ese crecimiento: pasar de 1 semana a 1 año es cambiar `FECHA_FIN` en la configuración

V	Cifras del dataset	¿Aplica?	Por qué
Velocidad	1 fichero al día, publicado con varios días o semanas de retraso	**No**	No hay flujo en tiempo real ni requisito de latencia por debajo del día. Por eso el procesamiento es batch (A.2: Lambda sin capa de velocidad)
Variedad	Un solo formato estructurado (CSV con `|`), 15 columnas fijas; tablas auxiliares también en CSV	**No**	Todo es tabular con un esquema conocido. Lo más heterogéneo son los códigos de zona, que mezclan códigos INE (`01001`), agregaciones de MITMA (`01004_AM`) y zonas extranjeras (`FR101`, `PT…`, `externo`, el 0,90 % de las filas). Lo resuelve una regla en la ETL
Veracidad	`viajes` es una **estimación** decimal a partir de telefonía móvil. Edad suprimida (`NA`) en el **30,2 %** de las filas y sexo en el **51,8 %**. 0 nulos y 0 filas inválidas	**Sí**	El problema no son errores de formato (las reglas de calidad no encontraron ninguno) sino la naturaleza del dato: son estimaciones con supresiones por privacidad. Por eso `NA` es una categoría y no un nulo, y los totales se leen como órdenes de magnitud, no como recuentos exactos
Valor	883,8 M de viajes estimados en la semana; efecto medible del festivo del 1-11 (D.2)	**Sí**	Permite planificar transporte, detectar caídas anómalas por municipio y modelar la demanda (E.3), que es el caso de uso de A.2

A.2 Caso de uso y elección de arquitectura

Elegimos un **Data Lake por zonas con procesamiento batch (arquitectura Lambda, solo con capa batch)**. El motivo es que la fuente llega una vez al día en ficheros y el caso de uso necesita poder reprocesar el histórico desde los datos crudos.

Caso de uso hipotético. El observatorio de movilidad de una consejería de transportes quiere dos cosas:

1. Un **panel que se actualiza cada mañana** con los viajes del último día publicado por provincia y tipo de día. Debe señalar los municipios cuya movilidad cae de forma anómala, por ejemplo en puentes y festivos.
2. **Consultas ad hoc sobre todo el histórico** y un modelo de demanda por municipio que se reentrena periódicamente. Por ejemplo: comparar Todos los Santos de 2022 con el de otro año, o añadir datos de población o de clima.

MITMA publica un fichero diario con varios días o semanas de retraso, y ha cambiado su metodología entre versiones del estudio.

Data Warehouse frente a Data Lake → Data Lake

	Data Warehouse	Data Lake (elegido)
Esquema	Al escribir: hay que modelar antes de cargar	Al leer en raw; esquema explícito en curated
Reprocesar si cambia la limpieza o la metodología	Requiere recargar desde fuera	Se reconstruye curated desde raw, que es inmutable
Nuevas fuentes (población, clima, eventos)	Nuevo modelado	Se añaden como ficheros en raw
ML sobre el dato fila a fila	Pensado para agregados	Acceso directo al detalle (70 M filas/semana)
Inconveniente	—	Sin gobierno se convierte en un "pantano": hacen falta calidad, cuarentena y esquema explícito

El inconveniente lo mitigamos con zonas (raw / curated / quarantine / results), un esquema declarado en curated y reglas de calidad. La zona results actúa como una pequeña capa de servicio, parecida a un DW, de la que leería el panel.

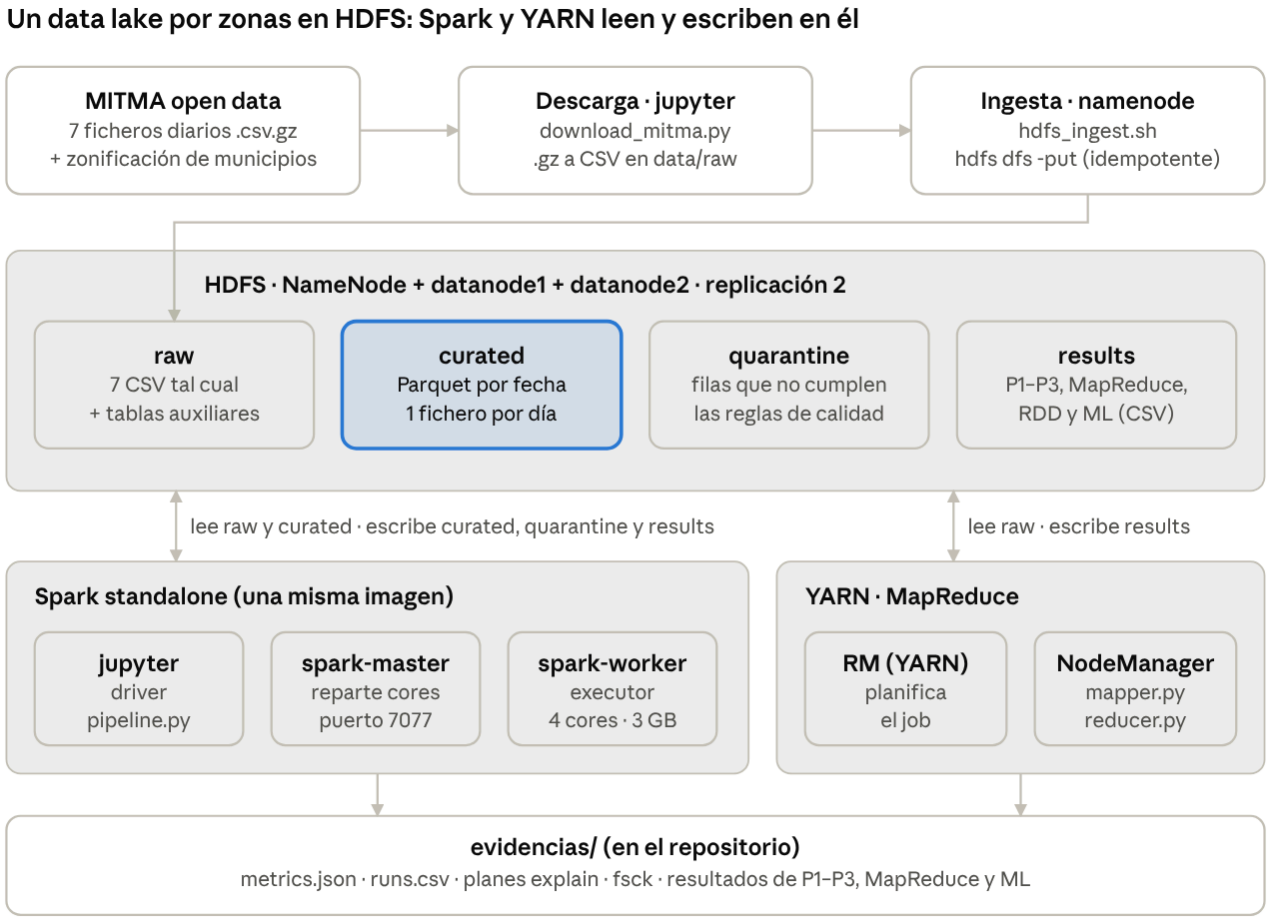
Lambda frente a Kappa → Lambda, solo con capa batch

	Lambda	Kappa
Idea	Capa batch (exacta) + capa de velocidad (tiempo real)	Todo es un stream que se reprocesa con *replay*
Encaje con MITMA	Los datos llegan en ficheros diarios. La capa batch cubre el panel diario y el histórico	No hay fuente en streaming: habría que simular un stream (Kafka) solo para reproducir ficheros

	Lambda	Kappa
Coste	Un solo pipeline mientras no exista la capa de velocidad	Infraestructura de streaming permanente sin necesidad
Inconveniente	Si se añade tiempo real, habría dos caminos de código que mantener	—

La latencia que pide el caso de uso es diaria y la exactitud del histórico es lo prioritario: por eso la capa batch de Lambda. La capa de velocidad queda como ampliación si un día llegaran datos de telefonía en tiempo real.

El diagrama muestra exactamente los servicios del `docker-compose.yml` y las zonas que crea el pipeline.



Arquitectura · fuente, ingesta, zonas de HDFS y los dos motores de cálculo

Los datos entran una vez por la ingesta y se quedan en HDFS. Spark (ETL, preguntas, RDD, ML) y YARN (MapReduce) leen de las zonas y escriben en ellas; las cifras acaban en `evidencias/`.

A.3 Entorno Docker reproducible

Un solo `docker compose up -d --build` levanta los 9 servicios en una única red de Docker. Desde un clon limpio no hace falta ningún paso manual: los contenedores se encuentran por nombre (`namenode` , `spark-master` , `jupyter`), sin el `docker network connect` del Tema 5.

Servicio	Imagen	Puertos	Función
namenode	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8	9870, 9000	Índice de HDFS. Monta el repo en `/opt/project` para la ingesta, `fsck` y MapReduce
datanode1, datanode2	bde2020/hadoop-datanode (misma versión)	—	Guardan los bloques. Replicación 2
resourcemanager	bde2020/hadoop-resourcemanager	8088	YARN: planifica el job de MapReduce
nodemanager	Imagen del curso + python3 (`docker/nodemanager/Dockerfile`)	—	Ejecuta las tasks de MapReduce (mapper.py, reducer.py)
historyserver	bde2020/hadoop-historyserver	—	Historial de jobs de YARN
spark-master	quay.io/jupyter/pyspark-notebook	8080	Master del cluster Spark standalone
spark-worker	quay.io/jupyter/pyspark-notebook	8081	Worker: 4 cores, 3 GB
jupyter	quay.io/jupyter/pyspark-notebook	8888, 4040	Entorno PySpark y driver del pipeline

Recursos. Docker tiene 8 GB, tal como pide el enunciado:

- **Spark:** worker de 4 cores y 3 GB, con un executor de 2 GB; driver de 2 GB.
- **YARN:** 4 GB y 4 vcores, con contenedores de map y reduce de 1 GB. En el `hadoop.env` del curso eran 16 GB, más de lo que tiene la máquina.

Spark y MapReduce nunca corren a la vez: el punto de entrada los lanza en secuencia.

Tres decisiones de diseño:

- **Misma imagen para el master, el worker y Jupyter.** Garantiza las mismas versiones de Spark y Python en el driver y en los executors; si no coinciden, PySpark falla en la primera acción.
- **NodeManager con Python.** En Hadoop Streaming, YARN arranca `mapper.py` y `reducer.py` como procesos dentro del NodeManager, y la imagen del curso no trae Python. Se añade python3 desde los repositorios archivados de Debian 9.
- **Repo montado en el NameNode.** Así `hdfs dfs -put` lee los datos directamente, sin copiar 7 GB con `docker cp` como en el Lab del Tema 5.

La versión exacta de Spark y Python que usó cada ejecución queda en `evidencias/metrics.json → a3_smoke`.

B.1 Zonas en HDFS y Parquet particionado

HDFS se organiza en cuatro zonas. Curated queda en **7 carpetas `fecha=…`, con un único fichero Parquet de 124–153 MB en cada una**, sin archivos pequeños.

Zona (HDFS)	Contenido	Cómo se escribe	Tamaño (datos / con réplicas)
`/raw/mitma/viajes`	Los 7 CSV diarios, sin modificar	`hdfs dfs -put` desde `scripts/hdfs_ingest.sh`; nunca se sobrescribe un fichero completo	6,4 GiB / 12,8 GiB
`/raw/mitma/municipios`, `/raw/mitma/aux`	Zonificación de MITMA; provincias y calendario (tablas propias)	Igual	< 100 KB
`/curated/mitma/viajes/fecha=YYYY-MM-DD`	Parquet limpio, tipado, con columnas derivadas	`etl.py`: `repartition(fecha)` + `partitionBy(fecha)` + overwrite dinámico	945,8 MiB (7 ficheros)
`/quarantine/mitma/viajes`	Filas que no cumplen las reglas	`overwrite`	1,8 KB: solo metadatos, 0 filas
`/results/mitma/...`	P1–P3, MapReduce, RDD, predicciones de ML	`overwrite`, `coalesce(1)` en CSV	KB

La estructura completa está en `evidencias/hdfs_ls_R.txt` (`hdfs dfs -ls -R`) y los tamaños en `hdfs_du.txt` y `hdfs_count.txt`.

Columna de partición: `fecha`, la inscrita. Con 7 días salen 7 particiones de unos 140 MB (media de 137 MB). Es un tamaño cercano al bloque de HDFS (128 MB), que es lo que recomendaba la inscripción: «particiones de cientos de MB, no de KB». Además, `fecha` es el filtro más frecuente: las consultas por día solo leen una carpeta (B.2). Particionar por provincia daría 52 carpetas de unos 19 MB, con un reparto muy desigual (Madrid frente a Soria).

Sin archivos pequeños. El `repartition(fecha)` antes de escribir pone todas las filas de un día en la misma task, y así sale un fichero por fecha. Sin él, las 32 particiones del shuffle de `dropDuplicates` escribirían 32 × 7 = 224 ficheros. Lo medimos en el desafío F.

Hallazgo en las evidencias. En la primera ejecución, `hdfs_du.txt` mostraba curated con **3 veces** su tamaño en disco (945,8 MiB → 2,8 GiB) y `fsck_curated.txt` marcaba 13 de 13 bloques como *under-replicated* («Target Replicas is 3 but found 2»). La causa: el cliente HDFS de Spark no lee el `dfs.replication = 2` del cluster y usa el valor por defecto de Hadoop, que es 3. Con 2 DataNodes no puede cumplirlo. Lo corregimos pasándolo en la

sesión (`spark.hadoop.dfs.replication = 2` , desde la configuración) y las evidencias finales ya muestran replicación 2 (B.2). Las salidas de la primera ejecución se conservan en `evidencias/hallazgo_replicacion_ejecucion1/`

B.2 Bloques, replicación y comparación de formatos

Bloques y replicación (`hdfs fsck`)

El fichero del 1-11 (868.895.935 bytes) está partido en **7 bloques**, y **cada bloque tiene sus 2 réplicas en los 2 DataNodes**: estado `HEALTHY` , replicación media 2,0 y 0 bloques *under-replicated*.

Se obtuvo con `hdfs fsck /raw/mitma/viajes/20221101_Viajes_municipios.csv -files -blocks -locations` , en `evidencias/fsck_raw_un_dia.txt` :

Dato del fsck	Valor	Interpretación
`868895935 bytes, replicated: replication=2, 7 block(s): OK`	7 bloques, factor 2	[868.895.935 / 134.217.728] = 7
Bloques 0–5	`len=134217728` cada uno	Bloques llenos de 128 MB (`dfs.blocksize` por defecto)
Bloque 6	`len=63589567` (60,6 MB)	El resto: 868.895.935 − 6 × 134.217.728. Un bloque a medio llenar solo ocupa su tamaño real en disco
Ubicaciones de cada bloque	`172.18.0.8:9866` y `172.18.0.4:9866`	Las dos réplicas en DataNodes distintos (`hdfs_dfsadmin_report.txt` : datanode1 = .8, datanode2 = .4). Con 2 nodos y factor 2, cada nodo tiene una copia completa del fichero
`Live_repl=2` en los 7 bloques	2	Ninguna réplica perdida

Dato del fsck	Valor	Interpretación
`Average block replication: 2.0`, `Under-replicated blocks: 0 (0.0 %)`, `Status: HEALTHY`	—	Se cumple el factor configurado

El orden de las ubicaciones cambia entre bloques (`.8, .4` o `.4, .8`), pero cada bloque está siempre en los dos nodos: la escritura pasa de un DataNode al otro en *pipeline* hasta completar el factor de replicación. Esto se repite para toda la zona raw: 6,4 GiB de datos ocupan 12,8 GiB en disco (`hdfs_du.txt`). El fsck de curated (`fsck_curated.txt`) muestra sus 7 ficheros Parquet en 13 bloques: los de más de 128 MB ocupan 2. Tras la corrección de la replicación, los 13 bloques tienen `replication=2`, `Under-replicated blocks: 0` y estado `HEALTHY`, y curated ocupa 945,8 MiB de datos y 1,8 GiB con réplicas: el doble exacto, como raw.

CSV frente a Parquet

El Parquet curado ocupa **6,9 veces menos** que el CSV. La misma consulta sobre todas las filas es **40 veces más rápida**, y filtrando un día, **43 veces**.

Método (`src/formats.py`):

- La misma consulta sobre raw (CSV, con el mismo esquema explícito) y sobre curated (Parquet).
- 3 ejecuciones de cada una; se usa la **mediana** porque la primera ejecución es más lenta.
- Sin `cache()`.
- Se ejecuta con `write.format("noop")`, que hace toda la consulta sin traer datos al driver, como en la clase 6.
- Dos consultas, porque miden efectos distintos:
  - **A:** suma de viajes por día. Lee todas las filas, pero solo 2 de 15 columnas.
  - **B:** suma de viajes por origen de un solo día.

	CSV (raw)	Parquet (curated)	Ratio
Tamaño en HDFS (sin réplicas)	6.871.662.808 B (6,87 GB)	991.783.251 B (0,99 GB)	6,9×
Particiones al leer	53	8	—
A · todas las filas, 2 columnas (3 ejecuciones)	41,97 / 40,50 / 39,82 s → mediana **40,50 s**	1,72 / 1,02 / 1,00 s → mediana **1,02 s**	40×

	CSV (raw)	Parquet (curated)	Ratio
B · un solo día (3 ejecuciones)	41,53 / 40,87 / 39,39 s → mediana **40,87 s**	1,61 / 0,94 / 0,69 s → mediana **0,94 s**	43×

Por qué.

1. **Formato columnar.** En la consulta A, Parquet lee solo las columnas `fecha` y `viajes` (en el plan, `ReadSchema: struct<...>`). El CSV tiene que leer y trocear las 15 columnas de cada línea y convertir cada número de texto a binario.
2. **Compresión.** Las columnas con pocos valores distintos (`edad`, `distancia`, `renta`) se comprimen muy bien, y de ahí el 6,9×. Es casi la misma proporción que en el Lab del T5 (175 MB → 26 MB).
3. **Partition pruning.** En la consulta B, el plan de Parquet muestra `PartitionFilters: [isnotnull(fecha), (fecha = 2022-10-26)]`: Spark abre solo la carpeta `fecha=2022-10-26` (`plan_b2_parquet_un_dia.txt`). El CSV no puede saltarse nada y tarda lo mismo con filtro que sin él (40,9 s frente a 40,5 s).

El problema de gzip, medido

Leer el mismo día en `.gz` da **1 partición** y en CSV descomprimido **8 particiones** (`b2_gzip_vs_csv_un_dia`). Un `.gz` no se puede empezar a leer por la mitad, así que toda su lectura la hace una sola task mientras los demás cores esperan. Contar sus filas tardó 3,23 s frente a 1,12 s. Por eso raw guarda el CSV descomprimido (decisión 5); lo descomprime el propio script de descarga.

C · MapReduce (Hadoop Streaming) frente a RDDs

Las dos implementaciones calculan los **viajes totales por provincia de origen y día** y dan **las mismas 364 claves** (52 provincias × 7 días). Ninguna clave aparece en una sola de las versiones, y la diferencia máxima es de 0,0005 viajes, el redondeo de la suma en coma flotante. MapReduce tarda **275 s** y la versión RDD **54,7 s** (mediana de 3), con 43 líneas de código frente a 8.

Implementaciones.

- **Hadoop Streaming** (`mapreduce/mapper.py`, `reducer.py`), lanzado en YARN por `scripts/mr_streaming.sh` con 2 reducers.
- **RDD** (`src/mr_rdd.py`): `textFile → filter(cabecera) → map(split) → filter(español) → map((provincia|fecha, viajes)) → reduceByKey(+)`, el patrón de la clase 4.

Las dos salidas están en `resultados/c_mr_streaming_provincia_fecha.tsv` y `c_rdd_provincia_fecha.tsv`.

Las tres fases con nuestros pares clave–valor

1. **Map.** Cada una de las **54 tasks de map** (una por split de ~128 MB de los 7 CSV) lee sus líneas por stdin. Por cada línea, por ejemplo `20221026|00|01001|01001|...|12.111|48.134`, emite el par `01|20221026 12.111`: provincia de origen + fecha como clave, viajes como valor. Descarta las 7 cabeceras y las filas con origen extranjero. Entraron 78.340.913 líneas (las 78.340.906 filas + 7 cabeceras) y salieron 77.980.522 pares, 1,41 GB.
2. **Shuffle + sort.** El *partitioner* asigna cada clave a uno de los 2 reducers según su hash, de modo que todas las líneas `01|20221026` de los 54 mappers acaban en el mismo reducer y **ordenadas**. Se movieron 108 bloques de salida (54 maps × 2 reducers) y se escribieron en disco 155.961.044 registros intermedios (*spilled records*).
3. **Reduce.** Cada reducer recorre sus líneas ordenadas y suma mientras la clave no cambia. Cuando cambia, emite `01|20221026 <total>`. Hubo 364 grupos y 364 líneas de salida (8.571 bytes).

Por qué MapReduce tarda 5 veces más

Factor	Hadoop Streaming	Spark RDD
Datos intermedios	Map → **disco** → shuffle → disco → reduce (156 M registros volcados)	El shuffle también escribe a disco, pero dentro de un stage todo va en memoria
Combinación antes del shuffle	Ninguna (sin `-combiner`): se envían **78 M de pares**	`reduceByKey` suma dentro de cada partición: ~364 pares por partición
Procesos	**Un contenedor YARN con su JVM por task**: 54 + 2 arranques	Executors que ya están arrancados y viven toda la aplicación
Java ↔ Python	Cada línea cruza dos veces por stdin/stdout entre la JVM y `python3`	También hay paso a Python (es un RDD de PySpark), pero por lotes y dentro del executor
Localidad	54 tasks *rack-local* y 0 *data-local*: el NodeManager no comparte contenedor con ningún DataNode	Igual: todo cruza la red de Docker

Factor	Hadoop Streaming	Spark RDD
Código	43 líneas: dos scripts, la cabecera y el cambio de clave a mano	8 líneas encadenadas

Cómo se midió.

- **RDD:** 3 ejecuciones completas, de 59,2 / 54,7 / 54,7 s. Cada repetición construye un RDD nuevo y lee los 6,4 GiB con 54 tasks (captura `c_sparkui_stages_rdd_reduceByKey.jpg`). En la primera ejecución del pipeline las repeticiones reutilizaban el shuffle del mismo RDD y solo medían el reduce (~1 s, con los stages de map *skipped*); lo corregimos.
- **MapReduce:** se mide con `date` alrededor de `hadoop jar` (`mr_streaming_time.json`). Su log completo de contadores está en `mr_streaming.log`, y el job aparece como `SUCCEEDED` en YARN (captura `c_yarn_jobs_mapreduce.jpg`).

La captura de los stages RDD muestra también el efecto de `reduceByKey`: cada stage de map lee 6,4 GiB pero solo escribe **175,6 KiB de shuffle**, frente a los 1,41 GB de salida del map de Hadoop Streaming.

D.1 ETL con esquema explícito

Las 78.340.906 filas de raw cumplen todas las reglas: **0 van a cuarentena, 0 son duplicadas y todos los orígenes españoles tienen nombre en la tabla auxiliar**. El curated conserva el 100 % de las filas y de los viajes (883,8 M).

El esquema se declara con `StructType` (15 columnas, `src/etl.py`) y nunca se usa `inferSchema`. Inferir obligaría a una pasada extra sobre 6,9 GB y convertiría `"01001"` en el entero 1001, perdiendo el 0 inicial del código INE.

Paso	Regla o transformación	Por qué	Filas antes	Filas después
1. Lectura	`StructType` + `header` + `sep="|"`	Tipos correctos sin adivinar; códigos como texto	—	78.340.906

Paso	Regla o transformación	Por qué	Filas antes	Filas después
2. Tipado	`fecha` texto → `to_date("yyyyMMdd")`; `periodo` → entero	Poder filtrar y particionar por fecha	78.340.906	78.340.906 (0 fechas nulas)
3. Nulos	`origen`, `destino`, `viajes`, `fecha`, `periodo` no nulos	Un flujo sin extremos o sin medida no significa nada	78.340.906	78.340.906
4. Rangos	`periodo` 0–23, `viajes` > 0, `viajes_km` ⩾ 0, `fecha` dentro de la semana	Valores imposibles o de otra semana	78.340.906	78.340.906 → **0 a cuarentena**
5. Duplicados	`dropDuplicates` sobre las 13 dimensiones	Un mismo trozo de flujo no puede contarse dos veces	78.340.906	78.340.906 (**0 duplicados**)
6. Zonas extranjeras	Marcadas (`origen_extranjero`, `destino_extranjero`), no eliminadas	Son datos válidos; cada análisis decide	704.364 filas (0,90 %) con algún extremo FR/PT/externo	Se conservan
7. Supresiones	`edad = NA` y `sexo = NA` como categoría	Supresión por privacidad, no dato ausente	23.674.348 (30,2 %) y 40.612.444 (51,8 %)	Se conservan
8. Derivadas	`cod_prov_origen/destino` = 2 primeros dígitos del código INE	Join con provincias sin tocar la tabla de MITMA	—	—

Que no salga ninguna fila inválida no significa que las reglas sobren: MITMA publica datos ya validados. Las reglas son la red que detecta un cambio en la fuente (otro separador, otro formato de fecha), y en ese caso el pipeline se detiene (umbral del 1 %). Con el modo

ANSI desactivado, un valor mal formado queda a `null` y acaba en cuarentena en vez de romper el job (decisión 16).

La tabla de municipios tiene 2.618 códigos, ninguno repetido, así que el join no duplica filas. Los 2.563 orígenes españoles distintos tienen todos su nombre. Sí detectamos un problema de codificación en la propia tabla de MITMA: algunos nombres llegan con caracteres corruptos (`PolinyÍ` por *Polinyà*). Lo dejamos tal cual porque raw es inmutable, y lo anotamos en Limitaciones.

La ETL tardó 779 s (`tiempo_etapa_etl_s`). La mayor parte es el shuffle de `dropDuplicates` sobre 78 M de filas.

D.2 Preguntas analíticas

Las tres preguntas van de una agregación a dos funciones de ventana. Las tres dicen lo mismo: **en un fin de semana o un festivo, España se mueve alrededor de un 20 % menos, y quien más cae son las provincias y municipios con más actividad laboral e industrial.**

P1 · Viajes medios diarios por provincia de origen y tipo de día (agrupación)

Tiene dos niveles de agregación: primero el total de viajes por provincia y día, después la media de esos totales según el tipo de día (tabla auxiliar `calendario`). Usa la API de DataFrames, con dos joins y `avg(when(...))` . Resultado: 52 filas, en `resultados/d2_p1_provincia_tipo_dia.csv` .

En el conjunto de España, un laborable medio genera **137,9 M de viajes**. El fin de semana baja a 112,7 M (**−18,3 %**) y el festivo del 1-11 a 105,9 M (**−23,2 %**). La mediana entre provincias es −16,9 % y −21,0 %.

Provincia	Media laborable	Fin de semana	Festivo 1-11	Finde vs lab.	Festivo vs lab.
Madrid	20,43 M	15,88 M	14,99 M	−22,3 %	−26,6 %
Barcelona	17,75 M	13,99 M	12,78 M	−21,2 %	−28,0 %
Valencia/València	7,43 M	6,01 M	5,50 M	−19,2 %	−26,0 %
Araba/Álava	0,96 M	0,74 M	0,68 M	−23,2 %	**−29,2 %**
Bizkaia	3,41 M	2,64 M	2,44 M	−22,6 %	−28,3 %
Zaragoza	2,64 M	2,03 M	1,98 M	**−23,3 %**	−25,0 %
Girona	2,41 M	2,19 M	2,02 M	−9,1 %	−16,2 %
Segovia	0,38 M	0,35 M	0,33 M	−7,8 %	−12,3 %
Ávila	0,37 M	0,36 M	0,35 M	**−2,1 %**	**−4,6 %**

Lectura. Las mayores caídas están en provincias industriales o con mucho trabajo de oficina (Álava, Bizkaia, Zaragoza, Madrid, Barcelona): sin trabajo, sus residentes hacen muchos menos viajes. Casi no caen las provincias de segunda residencia y turismo de interior cercanas a Madrid (Ávila, Segovia, Soria) o de la costa de Girona. Allí, el fin de semana genera tantos viajes como un laborable. La tabla completa de las 52 provincias está en el CSV.

P2 · Los 3 municipios que más viajes reciben cada día (ventana de ranking, Spark SQL)

Se hace con `SUM(viajes)` por (fecha, destino) y después `ROW_NUMBER() OVER (PARTITION BY fecha ORDER BY viajes_recibidos DESC)`, sobre vistas temporales. Resultado: 21 filas (7 días × 3), en `resultados/d2_p2_top_destinos_dia.csv`.

Día	1.º	2.º	3.º
mié 26-10	Madrid 10,67 M	Barcelona 5,53 M	València 2,53 M
jue 27-10	Madrid 10,67 M	Barcelona 5,46 M	València 2,54 M
vie 28-10	Madrid 10,45 M	Barcelona 5,39 M	València 2,55 M
sáb 29-10	Madrid 7,95 M	Barcelona 4,09 M	València 2,01 M
dom 30-10	Madrid 7,42 M	Barcelona 3,49 M	València 1,74 M
lun 31-10	Madrid 8,75 M	Barcelona 4,48 M	**Sevilla 2,15 M**
mar 01-11	Madrid 7,45 M	Barcelona 3,62 M	València 1,77 M

Lectura. El podio es estable, porque lo marca el tamaño de la ciudad e incluye los viajes dentro del propio municipio. Dos cosas se salen de la norma:

- **El lunes 31, víspera de festivo, Madrid recibe un 18 % menos que un laborable normal.** Es el «puente» que la tabla de calendario no marca como festivo (ver Limitaciones).
- **Ese mismo lunes, Sevilla adelanta a València.**

Usamos `ROW_NUMBER` y no `RANK` para garantizar exactamente 3 filas por día.

P3 · Desviación de cada municipio respecto a su media semanal (ventana de comparación + `lag`)

`avg(viajes_dia)` sobre `Window.partitionBy(destino)` da la media semanal de cada municipio sin colapsar filas, y `lag` sobre la misma ventana ordenada por fecha da la variación respecto al día anterior. Solo se miran los municipios que reciben al menos

20.000 viajes al día (`UMBRAL_MUNICIPIO_GRANDE`): 882 municipios × 7 días = 6.174 filas, en `resultados/d2_p3_desviacion_municipio_dia.csv`. Resumen por día:

Día	Mediana de la desviación	Mínimo	Máximo	Municipios > 20 % por debajo de su media
mié 26-10	+8,7 %	−40,0 %	+39,9 %	3
jue 27-10	+8,7 %	−39,2 %	+40,5 %	2
vie 28-10	+12,5 %	−27,7 %	+37,9 %	1
sáb 29-10	−4,4 %	−29,0 %	+30,3 %	7
dom 30-10	−14,0 %	−50,1 %	+45,8 %	**185**
lun 31-10	+2,4 %	−19,0 %	+38,6 %	0
mar 01-11	**−14,6 %**	−34,2 %	+34,8 %	**145**

Lectura. El festivo se comporta como un domingo: el municipio grande típico recibe un 15 % menos que en su semana media, y 145 de 882 quedan más de un 20 % por debajo. Las mayores caídas del 1-11 son municipios **industriales**: Almussafes (−34 %), Polinyà, Barberà del Vallès, Santa Perpètua de Mogoda, Riba-roja de Túria… Son polígonos que se vacían cuando la industria para (`resultados/d2_p3_mayores_caidas_festivo.csv`). El caso contrario lo marca Cocentaina, que celebra esos días su Fira de Tots Sants: +45,8 % el domingo y +34,8 % el festivo. El domingo también suben destinos de montaña y costa como Alp, Puigcerdà o Salou.

D.3 Plan de ejecución y optimización

En P1, el **broadcast join** baja la mediana de **70,4 s a 9,6 s (−86,3 %)** frente al sort-merge join, porque deja de mover **1.210 MB de shuffle** por la red.

Consulta elegida y por qué. P1 une los ~78 M de filas de curated con dos tablas diminutas: el calendario (7 filas) y las provincias (52 filas). Es el caso de la clase 6. Hay dos estrategias posibles:

- **Sort-merge join:** las dos tablas se reparten por la clave a través de la red y después se cruzan.

- **Broadcast join:** la tabla pequeña se copia entera a cada executor y la grande no se mueve.

Como Spark ya elige broadcast por su cuenta (umbral de 10 MB), medimos su efecto forzando el caso contrario. El «antes» usa `spark.sql.autoBroadcastJoinThreshold = -1`; el «después», el valor por defecto.

El plan interpretado. Los ficheros `plan_d3_*.txt` se leen de abajo arriba; cada `Exchange` es un shuffle y una frontera de stage.

	Antes: sort-merge join	Después: broadcast join
Joins en el plan	2 × `SortMergeJoin Inner`	2 × `BroadcastHashJoin Inner BuildRight`
Cómo llegan los datos al join	`Exchange hashpartitioning` en **cada lado** de cada join (4 shuffles) + `Sort`	`BroadcastExchange` solo de la tabla pequeña; curated no se mueve
Shuffles de la agregación	2 (`groupBy` por provincia y día, y después por provincia)	Los mismos 2, con muy pocos datos (agregación parcial)
Stages ejecutados (API de la Spark UI)	9	7
Shuffle write total	**1.210,1 MB** (637,2 MB en el primer join + 572,9 MB en el segundo)	**< 0,1 MB**
Lectura de curated	263,7 MB, 8 tasks	263,7 MB, 8 tasks
Tiempo (3 ejecuciones)	73,2 / 45,2 / 70,4 s → mediana 70,4 **s**	9,7 / 9,4 / 9,6 s → mediana 9,6 **s**

Lectura. Las dos versiones leen lo mismo: 263,7 MB, es decir, solo las 4 columnas que usa P1 de un Parquet de 0,99 GB, gracias al formato columnar. Toda la diferencia está en el shuffle: con sort-merge, los ~78 M de filas se escriben en disco y viajan por la red dos veces, una por join, para encontrarse con 7 y 52 filas. Con broadcast, esas 59 filas viajan a cada executor y los datos grandes no se mueven. El sort-merge varía mucho entre ejecuciones (45–73 s): sus 1,2 GB de shuffle pasan por el disco del worker, y el resultado depende de lo que quede en la caché del sistema operativo. El broadcast es estable (9,4–9,7 s). Por eso usamos la mediana. Las capturas de la pestaña SQL de la Spark UI están en `evidencias/capturas/d3_*` (ver su `LEEME.md`).

Lo que añade la Spark UI. El plan final que muestra la pestaña SQL incluye las estadísticas reales de cada `ShuffleQueryStage`, porque Spark 4 ejecuta con *Adaptive Query Execution* (`AdaptiveSparkPlan`, `AQEShuffleRead ... coalesced`):

- **Sort-merge** (`d3_sparkui_plan_antes_sortmerge.jpg`): el lado de curated entra en el primer join como un shuffle de **78 M de filas y 2,9 GiB** y en el segundo como otro de **78 M de filas y 4,6 GiB** (en memoria; en disco, comprimidos, son los 1.210 MB de la tabla).
- **Broadcast** (`d3_sparkui_plan_despues_broadcast.jpg`): solo viajan los dos `BroadcastQueryStage`, de 7 y 52 filas. El primer shuffle que aparece ya es el de la agregación: 660 filas y 58,3 KiB.

AQE no convierte el sort-merge en broadcast por su cuenta porque lo desactivamos explícitamente (umbral −1). Por eso las dos versiones son comparables. Las capturas se tomaron en una ejecución aparte de P1 con el mismo código y la misma tabla curada, porque la Spark UI de una aplicación desaparece cuando termina el pipeline.

Las cifras de stages y bytes de shuffle salen de la API REST de la propia Spark UI (`/api/v1/applications/.../stages`, en `metrics.json → d3_optimizacion_broadcast`). Son los mismos datos que muestra la pestaña Stages.

E.1–E.2 Integración del pipeline

Un único comando ejecuta todo el flujo: `.\run_pipeline.ps1` en Windows o `./run_pipeline.sh` en Linux/macOS. Cada paso corre dentro de su contenedor; el script solo los encadena y se detiene en el primer error.

Paso	Contenedor	Qué hace
0	host	`docker compose up -d --build`: no hace nada si el entorno ya está levantado
1	jupyter	`scripts/download_mitma.py`: descarga los `.csv.gz` que falten y los descomprime
2	namenode	`scripts/hdfs_ingest.sh`: `hdfs dfs -put` a raw
3	jupyter → Spark	`src/pipeline.py`: smoke, etl, formats, analytics
4	namenode → YARN	`scripts/mr_streaming.sh`: Hadoop Streaming
5	jupyter → Spark	`src/pipeline.py`: mr_rdd, ml, challenge
6	namenode	`scripts/hdfs_evidence.sh`: `ls -R`, `du`, `count`, `fsck`

Configuración. Todas las rutas y parámetros están en un solo fichero, `config/pipeline.env`:

- zonas de HDFS;
- fechas de la semana y URL de descarga;
- separador, formato de fecha y columna de partición;
- umbrales de calidad;
- recursos de Spark;
- parámetros del modelo.

Elegimos el formato `.env` porque lo leen tanto los scripts bash de los contenedores Hadoop (`source`, sin necesidad de un parser de YAML en Debian 9) como el código Python (`load_config()`, que devuelve un diccionario CONFIG como el de la clase del Tema 5). Ningún fichero de `src/`, `scripts/` o `mapreduce/` contiene rutas de HDFS. El mapper recibe incluso el separador desde la configuración, con `-cmdenv`.

Idempotencia. Ejecutar el pipeline dos veces deja exactamente el mismo resultado. Cada zona se escribe así:

Zona	Estrategia de escritura	Por qué es idempotente
raw	`put` solo si el fichero no existe o su tamaño no coincide	Raw es inmutable: nunca se sobrescribe un dato correcto
curated	`mode("overwrite")` + `partitionOverwriteMode=dynamic` + `repartition(fecha)`	Se reemplazan solo las fechas presentes; reprocesar un día no borra los demás (Demo 2 del Tema 5)
quarantine, results	`mode("overwrite")`	Cada ejecución reemplaza su salida completa
salida de MapReduce	`hdfs dfs -rm -r` antes del job	Hadoop exige que la salida no exista

Evidencia: `evidencias/runs.csv` añade una línea por ejecución de la ETL. Las dos ejecuciones completas (12:42 y 13:44 UTC) dejan exactamente los mismos conteos: 78.340.906 filas en raw, 0 en cuarentena, 78.340.906 en curated y 883.809.802,6 viajes. La segunda ejecución no volvió a subir nada a raw (la ingesta detectó los 7 ficheros con el mismo tamaño), y curated sigue teniendo 7 ficheros, no 14.

Controles de calidad con acción definida

Control	Regla	Acción
Filas inválidas	Fecha fuera de la semana o no parseable, `periodo` fuera de 0–23, origen o destino nulos, `viajes` ⩽ 0 o nulo, `viajes_km` < 0	**Cuarentena** (`/quarantine`)
Proporción de inválidas	> 1 % (`MAX_BAD_RATIO`)	**Detener** el pipeline (`assert` , como el "Fail loudly" del Tema 5)
Duplicados	Misma clave natural (las 13 dimensiones del flujo)	**Eliminar y registrar** el número
Clave de la tabla auxiliar	`id_municipio` repetido	**Registrar** (un join con claves repetidas duplica filas)
Municipios sin nombre	> 1 % de orígenes españoles sin match	**Avisar**
Nulos y supresiones	Nulos por columna, `edad` / `sexo` = NA	**Registrar** en `metrics.json`

E.3 Línea base de Machine Learning (MLlib)

La regresión lineal municipio + edad explica el **82,4 % de la varianza** en test (R^2 = 0,824, RMSE = 0,676 en escala log). La media global se queda en R^2 = 0 y la media del municipio en R^2 = 0,524: **el modelo supera a las dos baselines**, y la diferencia con la segunda es lo que aporta la edad.

El problema. Regresión lineal multivariable:

- **Objetivo:** `log1p` de los viajes medios diarios que salen de un municipio para un rango de edad.
- **Entradas:** municipio de origen y rango de edad, ambas categóricas.
- Sin variables de tiempo (decisión 13).

Paso	Implementación (`src/ml.py`)
Dataset	`groupBy(origen, edad)` → viajes / 7 días, solo orígenes españoles: **12.815 filas** (2.561 municipios × ⩽5 rangos)
Objetivo	`log1p(viajes_dia)` : entre un pueblo y Madrid hay 5 órdenes de magnitud

Paso	Implementación (`src/ml.py`)
Partición	`randomSplit([0.8, 0.2], seed=42)` → 10.266 train / 2.549 test
Codificación	`StringIndexer` + `OneHotEncoder` en las dos variables, `handleInvalid="keep"` (un municipio que solo está en test no rompe el modelo)
Features	`VectorAssembler` → vector disperso de 2.568 posiciones
Modelo	`LinearRegression(regParam=0.01)` , dentro de un `Pipeline` ajustado **solo con train** (los codificadores tampoco ven el test)
Métricas	`RegressionEvaluator` : RMSE, MAE, R^2 sobre test

Modelo (test)	RMSE	MAE	R^2
Baseline 1: media global de train	1,612	1,322	0,000
Baseline 2: media del municipio en train	1,112	0,835	0,524
Regresión lineal municipio + edad	**0,676**	**0,473**	**0,824**

Lectura. Un MAE de 0,47 en escala log equivale a fallar, de media, por un factor de unas 1,6 veces (e^0,47) en los viajes. Es razonable para un modelo que solo conoce el municipio y la edad. Los coeficientes de edad (`metrics.json → e3_ml.coeficientes_edad`) son coherentes:

- `65-100` es el rango que menos viajes genera (−0,87 respecto al resto).
- `NA` es el que más (+1,24), porque agrupa los segmentos suprimidos por privacidad, que suman muchos viajes.

El modelo es aditivo: no puede aprender que el perfil de edad cambie de un municipio a otro. Como pide el enunciado, es una línea base, sin ajuste de hiperparámetros. Las predicciones están en `resultados/e3_predicciones_test.csv` .

F · Desafío 3: el problema de los archivos pequeños

Escribir curated tras un shuffle de 200 particiones genera **1.400 ficheros de 0,9 MB**. Leerlos es **6,5 veces más lento** que leer los **7 ficheros de 137 MB** que deja `repartition("fecha")` , y escribirlos, 1,5 veces más lento.

El problema. Con `partitionBy("fecha")`, cada task escribe un fichero en cada carpeta de fecha de la que tenga filas. Después de una transformación ancha (`dropDuplicates`, un join…), cada partición contiene filas de los 7 días, así que se generan particiones × 7 ficheros. Cada fichero es:

- un objeto más en la memoria del NameNode;
- al menos una task más que planificar, abrir y cerrar al leer.

Experimento (`src/challenge.py`). La misma tabla curada (78,3 M filas, 0,99 GB) se escribe de tres formas en `/tmp/mitma_smallfiles`. Para cada una se mide la escritura y una lectura completa (suma de viajes por día, mediana de 3):

Variante	Ficheros	Tamaño medio	Tareas al leer	Escritura	Lectura (3 ejecuciones → mediana)
`repartition(200)`: `shuffle.partitions` por defecto de Spark	**1.400**	0,9 MB	54	429 s	10,18 / 5,74 / 5,59 s → **5,74 s**
`repartition(32)`: `shuffle.partitions` del proyecto	224	5,2 MB	17	468 s	1,95 / 1,72 / 1,36 s → 1,72 s
`repartition("fecha")`: lo que hace `etl.py`	**7**	137 MB	8	286 s	0,88 / 3,22 / 0,52 s → **0,88 s**

Lectura.

- **Escritura:** con 1.400 ficheros, cada uno pide su creación al NameNode, abre un *pipeline* de replicación hacia los DataNodes y escribe sus metadatos Parquet. Con 7, ese coste fijo se paga 7 veces.
- **Lectura:** Spark agrupa ficheros pequeños en tasks (54 en vez de 1.400), pero sigue abriendo 1.400 ficheros y leyendo 1.400 *footers* de Parquet. Además, con ficheros de 0,9 MB la compresión por columnas y las estadísticas de cada bloque de filas sirven de poco.
- `hdfs_count.txt` cuenta los 3 experimentos juntos: 1.634 ficheros, que son 1.400 + 224 + 7 + 3 `_SUCCESS`.

El tiempo de escritura no ordena las variantes de forma limpia: la de 32 particiones tardó más que la de 200. Las tres pagan el mismo shuffle completo de los 78 M de filas (5,2–5,5 GiB de *shuffle write* por variante, según la pestaña Stages de la Spark UI), que domina el tiempo y varía entre ejecuciones. El efecto robusto está en la lectura, que es lo que se repite en cada consulta. Las cifras de la primera ejecución van en la misma dirección (`hallazgo_replicacion_ejecucion1/metrics.json`: 8,73 s frente a 0,69 s).

Lo que nos llevamos al diseño. Particionar es una decisión de rendimiento que se toma al escribir (T5). El `repartition` por la columna de partición justo antes del `write` cuesta un shuffle, pero lo compensa en la primera lectura. El extremo opuesto también es malo: con unos 140 MB por día, particionar más fino (por día y provincia) volvería a dar ficheros de pocos MB.

Limitaciones

- **Una sola semana.** Las medias por tipo de día se apoyan en 4 laborables, 2 días de fin de semana y 1 festivo. Los porcentajes de P1 y P3 describen esta semana concreta, no un patrón estable.
- **El lunes 31 de octubre es víspera de festivo.** Lo clasificamos como laborable, pero en muchos sectores fue puente: rebaja la media laborable.
- **Sin localidad de datos.** HDFS y Spark viven en contenedores distintos de una misma máquina: cada lectura cruza la red de Docker. Los tiempos no son representativos de un cluster real, donde el executor lee el bloque de su propio disco.
- **Un solo worker y un solo NodeManager.** Las comparaciones de tiempo (MapReduce/RDD, CSV/Parquet, joins) miden el coste relativo, no la escalabilidad.
- **MapReduce va en un paso aparte del punto de entrada.** Se lanza en el contenedor `namenode`, que es el que tiene el cliente de Hadoop. Sigue dentro del único comando, pero no dentro de la sesión de Spark.
- **NodeManager con repositorios archivados.** Python 3.5 se instala de `archive.debian.org` con `[trusted=yes]` porque las claves de Debian 9 caducaron. Es aceptable en un laboratorio local, no en producción.
- **Los datos de MITMA son estimaciones** a partir de telefonía móvil, con supresiones por privacidad (edad y sexo = NA). Los totales no son conteos exactos de personas.
- **El modelo de ML es una línea base.** Es aditivo (municipio + edad, sin interacción), sin variables temporales y con un R^2 dominado por el tamaño del municipio. No sirve para predecir picos de un día concreto.
- **Seguridad de laboratorio.** `HADOOP_USER_NAME=root`, permisos de HDFS desactivados y Jupyter con un token fijo, como en las clases.

Registro de decisiones de diseño

#	Decisión	Alternativas	Motivo
1	Dataset: **1 semana** (26-10 a 01-11-2022) en lugar de 2 meses	2 meses inscritos	~7 GB caben con holgura en el entorno local (8 GB de RAM, 31 GB de disco) y siguen dentro de los 5–20 GB exigidos
2	Nueva **P1**: media diaria por provincia y **tipo de día** (laborable / fin de semana / festivo)	P1 inscrita: por día de la semana	Con una semana, cada día de la semana aparece una sola vez (n = 1). La semana incluye el festivo nacional del 1 de noviembre
3	**Tabla auxiliar de calendario** construida por el equipo	Calcular el tipo de día en el código	El enunciado permite construir tablas auxiliares; el festivo es un dato, no una regla
4	**Tabla de provincias** (código INE → provincia, CCAA)	Ampliar la tabla de municipios	La provincia son los 2 primeros dígitos del código INE: 52 filas bastan y no hay que tocar la tabla de MITMA
5	**Raw guarda el CSV descomprimido**	Guardar el `.gz` y hacer `repartition` al leer	gzip no es divisible: 1 fichero = 1 task. Medido en B.2
6	Partición de curated por **fecha** (columna inscrita) + `repartition(fecha)`	Particionar por provincia o por mes	7 particiones de un tamaño razonable, 1 fichero por día. Es el filtro más habitual
7	Zonas extranjeras (FR…, PT…, externo): **se conservan y se marcan** (`origen_extranjero` , `destino_extranjero`)	Eliminarlas en la ETL	Son datos válidos; cada análisis decide si los usa. Raw y curated no pierden información

#	Decisión	Alternativas	Motivo
8	`edad = NA` y `sexo = NA` como **categoría**	Imputar o eliminar	Es una supresión de MITMA por privacidad, no un dato ausente. Eliminarlas quitaría alrededor del 30 % de las filas
9	Configuración en `.env`	YAML o JSON	La leen bash (contenedores Hadoop) y Python sin dependencias
10	MapReduce: **provincia de origen × fecha**, sin combiner	Por municipio, con combiner	364 claves, una agregación con sentido sin join. Sin combiner se ve mejor la diferencia con `reduceByKey`
11	D.3: **broadcast join frente a sort-merge**	Cache, partition pruning	P1 une 70 M de filas con dos tablas pequeñas: es el caso de libro de la clase 6
12	ML: **viajes medios diarios por (municipio de origen, edad)**, en escala `log1p`	Fila cruda, destino, escala lineal	En la fila cruda la variación la explica lo que no está en el modelo (destino, hora, distancia). En lineal mandarían Madrid y Barcelona
13	Partición train/test **aleatoria** (80/20, semilla 42)	Cronológica	El modelo no usa variables de tiempo (decisión del equipo); cada fila ya es una media semanal
14	Desafío **F.3 archivos pequeños**	Tolerancia a fallos, calidad, AQE	Se conecta con la decisión 6 y se mide con el mismo método que B.2
15	Punto de entrada en **PowerShell y bash**	Solo bash	El equipo trabaja en Windows y en macOS

#	Decisión	Alternativas	Motivo
16	spark.sql.ansi.enabled = false	Modo ANSI (por defecto en Spark 4)	La imagen actual trae Spark 4.2. Con ANSI, un valor mal formado (to_date inválido) lanza una excepción y para el job; sin ANSI queda a null, como en el Spark 3.5 de clase, y las reglas de calidad lo mandan a cuarentena

Trazabilidad: dónde está cada cifra

Todas las cifras de este documento salen de la **segunda ejecución completa** de `run_pipeline.ps1` (20-10-2026, de 15:30 a 16:23, transcripción en `evidencias/pipeline_run.log`), en el entorno Docker descrito en A.3, y están en el repositorio. Esa ejecución ya incluye las tres correcciones de código: replicación 2, modo ANSI y medición de los RDD. Las salidas de la primera ejecución que motivaron las correcciones se conservan en `evidencias/hallazgo_replicacion_ejecucion1/`.

Sección	Evidencia	Generada por
A.1, D.1	`evidencias/metrics.json` → `etl_1_entrada`, `etl_2_cuarentena`, `etl_3_curated`, `etl_4_tabla_auxiliar`	`src/etl.py`
A.3	`metrics.json` → `a3_smoke`; `evidencias/capturas/`	`src/pipeline.py` (smoke)
B.1	`evidencias/hdfs_raw_ls.txt`, `hdfs_ls_R.txt`, `hdfs_count.txt`, `hdfs_du.txt`	`scripts/hdfs_ingest.sh`, `scripts/hdfs_evidence.sh`
B.2	`fsck_raw_un_dia.txt`, `fsck_curated.txt`; `metrics.json` → `b2_formatos`, `b2_gzip_vs_csv_un_dia`; `plan_b2_*.txt`	`scripts/hdfs_evidence.sh`, `src/formats.py`
C	`mr_streaming.log`, `mr_streaming_time.json`, `resultados/c_mr_streaming_provincia_fecha.tsv`, `resultados/c_rdd_provincia_fecha.tsv`; `metrics.json` → `c_mapreduce_vs_rdd`	`scripts/mr_streaming.sh`, `src/mr_rdd.py`
D.2	`resultados/d2_p1_*.csv`, `d2_p2_*.csv`, `d2_p3_*.csv`; `plan_d2_p1/p2/p3.txt`	`src/analytics.py`

Sección	Evidencia	Generada por
D.3	`metrics.json` → `d3_optimizacion_broadcast`; `plan_d3_antes_sort_merge_join.txt`, `plan_d3_despues_broadcast_join.txt`; capturas de la Spark UI	`src/analytics.py`
E.2	`evidencias/runs.csv` (una línea por ejecución)	`src/etl.py`
E.3	`metrics.json` → `e3_ml`; `resultados/e3_predicciones_test.csv`	`src/ml.py`
F	`metrics.json` → `f3_archivos_pequenos`; `hdfs_count.txt`	`src/challenge.py`

Los tiempos dependen de la máquina. Los conteos y los resultados de las consultas son deterministas: cualquiera que clone el repositorio y ejecute el pipeline obtiene los mismos.